Be Careful with Migrations

Migrations are inherently risky—this applies to Ecto, ActiveRecord, and any other migration framework. The risk comes from the fact that we're changing the schema of a running database. There's nothing that's stopping us from running `drop table(:something_important)` and losing all of our data, so we need to be careful with migrations.

The Fly.io blog post[15] about safe migrations can help you understand the risk that you face, and it offers tips for how to mitigate that risk. You can get away without knowing these dangers if you have a small amount of data, but a database with more data and throughput increases the migration risk.

One final note on this topic is that—similar to ActiveRecord—Ecto migrations have built-in protections to ensure that your database doesn't end up in an invalid state. The first protection is a DDL transaction that runs around the entire migration file. This transaction ensures that if an error occurs during different statements in our migration file, then the entire migration is rolled back and the database is unaffected. The second protection is a migration lock that's taken out on the `schema_migrations` table. This ensures that two migrations are not executed at the same time, which increases the chance for errors to occur.

You can disable these locks for certain operations, but you should rarely need to worry about them. They are there for protection, so disable them only when absolutely necessary. The most common operation that requires disabling them is when you add an index with the `concurrently: true` option.

Next, let's look at how to persist data in our database.

Use Changesets to Persist Data

Now that we have schemas, we need a way to insert data into the database. The approach that Ecto takes is significantly different than ActiveRecord, but it's one of the most loved components of Ecto. Let's dive into Ecto changesets.

`Ecto.Changeset`[16] lets us turn data parameters into a persisted database record. Changesets are used for creating new records or updating existing records. We can also validate incoming data using changesets—this ensures our database records match our application's business rules.

Let's write our first changeset. We'll start with the `Contact` schema because it has fewer fields.

15. https://fly.io/phoenix-files/safe-ecto-migrations/
16. https://hexdocs.pm/ecto/Ecto.Changeset.html